Name of Student: Vyankatesh Rohokale	**PRN: 26070122518**
Semester: III	**Year: AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No: 25	
TITLE: Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.	

Problem Statement

Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Learning Objectives

To create and manage multiple threads using the Thread class.

Programme

1.Create a program where multiple threads print numbers simultaneously

```
package assignment25;
public class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {

System.out.println(Thread.currentThread().getName() + ": " +
i);
        }
    }

    public static void main(String[] args) {
        MyThread obj1 = new MyThread();
        MyThread obj2 = new MyThread();

        obj1.start();
        obj2.start();
    }
}
```

```
piyushrohokale@Piyushs-MacBook-Air assignment25 % javac MyThread.java
piyushrohokale@Piyushs-MacBook-Air assignment25 % java MyThread
Thread-0: 0
Thread-1: 0
Thread-0: 1
Thread-0: 2
Thread-0: 3
Thread-0: 4
Thread-1: 1
Thread-1: 2
Thread-1: 3
Thread-1: 4
piyushrohokale@Piyushs-MacBook-Air assignment25 %
```

2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

```
class DepositThread extends Thread {
    static int balance = 10000;

    public void run() {
        for (int i = 0; i < 5; i++) {
            deposit(500);
        }
    }

    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount + " |
Balance: " + balance);
    }
}

class WithdrawThread extends Thread {
    public void run() {
        for (int i = 0; i < 5; i++) {
            withdraw(300);
        }
    }

    synchronized void withdraw(int amount) {
        if (DepositThread.balance >= amount) {
            DepositThread.balance -= amount;
```

```
            System.out.println("Withdrawn: " + amount + " |
Balance: " + DepositThread.balance);
        } else {
            System.out.println("Insufficient balance");
        }
    }
}

public class BankTransaction {
    public static void main(String[] args) {
        DepositThread obj1 = new DepositThread();
        WithdrawThread obj2 = new WithdrawThread();

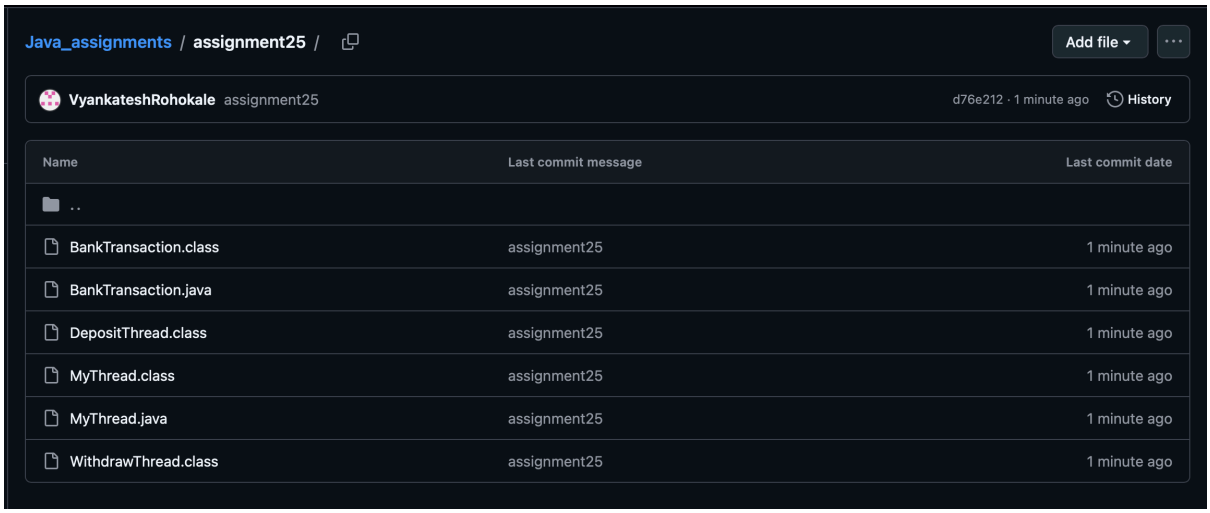
        obj1.start();
        obj2.start();
    }
}
```

```
piyushrohokale@Piyushs-MacBook-Air assignment25 % javac BankTransaction.java
piyushrohokale@Piyushs-MacBook-Air assignment25 % java BankTransaction
Withdrawn: 300 | Balance: 10200
Deposited: 500 | Balance: 10500
Withdrawn: 300 | Balance: 9900
Deposited: 500 | Balance: 10400
Withdrawn: 300 | Balance: 10100
Withdrawn: 300 | Balance: 10300
Withdrawn: 300 | Balance: 10000
Deposited: 500 | Balance: 10600
Deposited: 500 | Balance: 10500
Deposited: 500 | Balance: 11000
piyushrohokale@Piyushs-MacBook-Air assignment25 %
```

Conclusion:

The first program demonstrates concurrent execution by printing numbers from multiple threads, while the second simulates bank deposit and withdrawal operations using threads and displays transaction details.

Github:


Java_assignments / assignment25 /
Add file
VyankateshRohokale assignment25
d76e212 · 1 minute ago
History
Name
Last commit message
Last commit date
..
BankTransaction.class assignment25 1 minute ago
BankTransaction.java assignment25 1 minute ago
DepositThread.class assignment25 1 minute ago
MyThread.class assignment25 1 minute ago
MyThread.java assignment25 1 minute ago
WithdrawThread.class assignment25 1 minute ago